

Paid Text Embedding Models – Complete Usage Guide

This README explains **top paid embedding models**, their pricing, and **how to install, authenticate, and use each model** in Python. These embeddings are commonly used in **RAG pipelines, vector databases, semantic search, and recommendation systems**.

What Are Embeddings?

Embeddings convert text into **high-dimensional vectors** that capture semantic meaning. Similar texts produce similar vectors, enabling: - Semantic search - Retrieval-Augmented Generation (RAG) - Clustering & recommendation - Document similarity

All paid models work via **API access** and are priced by **input tokens or characters**.

Top 7 Paid Embedding Models (2025)

Rank	Model	Provider	MTEB	Price / 1M	Dimensions	Used By
1	text-embedding-3-large	OpenAI	64.6	\$0.13	3072 / 1536	Canva, Shopify, Spotify
2	voyage-3-large	Voyage AI	66.8	\$0.12	1536	MongoDB, Anthropic
3	embed-v4	Cohere	65.2	\$0.10	1024	AWS, Enterprises
4	textembedding-gecko	Google Vertex AI	~62	~\$0.10	768	Google Cloud RAG
5	titan-embed-text-v2	Amazon Bedrock	~61	~\$0.11	1024 / 1536	AWS Enterprises
6	mistral-embed	Mistral AI	~60	\$0.10	1024	Mistral API Users
7	arctic-embed-l	Snowflake	~59	\$0.04	1024	Snowflake / Weaviate

Common Workflow (All Paid APIs)

1. Create account on provider website

2. Generate API key from dashboard
 3. Install SDK via `pip`
 4. Set API key as environment variable
 5. Send text to **embedding endpoint**
 6. Store vectors in vector DB (FAISS, Pinecone, Weaviate, Chroma, etc.)
-

1 OpenAI – text-embedding-3-large

Installation

```
pip install openai
```

Get API Key

<https://platform.openai.com/api-keys>

Usage

```
import os
from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def openai_embed(texts):
    response = client.embeddings.create(
        model="text-embedding-3-large",
        input=texts
    )
    return [e.embedding for e in response.data]
```

2 Voyage AI – voyage-3-large

Installation

```
pip install langchain-voyageai
```

Get API Key

<https://www.voyageai.com>

Usage

```
import os
from langchain_voyageai import VoyageAIEmbeddings

os.environ["VOYAGE_API_KEY"] = "your_api_key"

embeddings = VoyageAIEmbeddings(model="voyage-3-large")

def voyage_embed(texts):
    return embeddings.embed_documents(texts)
```

3 Cohere – embed-v4

Installation

```
pip install langchain-cohere cohere
```

Get API Key

<https://dashboard.cohere.com>

Usage

```
import os
from langchain_cohere import CohereEmbeddings

os.environ["COHERE_API_KEY"] = "your_api_key"

embeddings = CohereEmbeddings(model="embed-english-v4.0")

def cohere_embed(texts):
    return embeddings.embed_documents(texts)
```

4 Google Vertex AI – textembedding-gecko

Setup

- Enable **Vertex AI API** in Google Cloud Console

- Create Service Account & JSON key
- Set `GOOGLE_APPLICATION_CREDENTIALS`

Installation

```
pip install google-cloud-aiplatform
```

Usage

```
import vertexai
from vertexai.language_models import TextEmbeddingModel

vertexai.init(project="your-project-id", location="us-central1")

model = TextEmbeddingModel.from_pretrained("textembedding-gecko@001")

def google_embed(texts):
    embeddings = model.get_embeddings(texts)
    return [e.values for e in embeddings]
```

5 Amazon Bedrock – titan-embed-text-v2

Setup

- AWS account required
- Enable Amazon Bedrock
- Create IAM role with Bedrock access

Installation

```
pip install boto3 langchain-aws
```

Usage

```
from langchain.embeddings import BedrockEmbeddings

bedrock = BedrockEmbeddings(
    model_id="amazon.titan-embed-text-v2"
)
```

```
def amazon_embed(texts):  
    return bedrock.embed_documents(texts)
```

6 Mistral AI – mistral-embed

Installation

```
pip install mistralai
```

Get API Key

<https://console.mistral.ai>

Usage

```
import os  
from mistralai import Mistral  
  
client = Mistral(api_key=os.getenv("MISTRAL_API_KEY"))  
  
def mistral_embed(texts):  
    response = client.embeddings.create(  
        model="mistral-embed",  
        inputs=texts  
    )  
    return [d.embedding for d in response.data]
```

7 Snowflake – arctic-embed-l

Access Methods

- Snowflake Cortex AI
- Weaviate + Snowflake

Installation

```
pip install snowflake-connector-python
```

SQL Usage (Cortex)

```
SELECT SNOWFLAKE.CORTEX.EMBED_TEXT(  
  'snowflake-arctic-embed-1',  
  'Your text here'  
);
```

Python Usage (via Snowflake Query)

```
import snowflake.connector  
  
conn = snowflake.connector.connect(  
  user="USER",  
  password="PASSWORD",  
  account="ACCOUNT"  
)
```

Choosing the Right Model

Use Case	Recommended Model
Best overall quality	Voyage 3 Large
Best OpenAI ecosystem	text-embedding-3-large
Cheapest enterprise	Snowflake Arctic
AWS-native RAG	Titan Embed
Google Cloud RAG	Gecko
Open-source friendly	Mistral Embed

Production Tips

- Normalize vectors before storage
- Use batch embedding for cost efficiency
- Match vector dimension with DB index
- Cache embeddings aggressively
- Track token usage & cost

License & Cost Disclaimer

Pricing and performance may change. Always verify on official provider documentation.

✅ This README is production-ready and suitable for GitHub RAG / embedding projects.